# Clase 8 · La FPU de la Pico 2 en ensamblador

Organización y Arquitectura de Computadoras · M.I. José Antonio de Jesús Arredondo Garza · miércoles 30 sep 2026 · Tema 3 · Apuntes y resumen de estudio

## PARTE 1 · Apuntes completos

### 1. Banco de registros y límites de la FPU

- La Unidad de Punto Flotante del RP2350 tiene **32 registros de 32 bits, s0 a s31**, cableados para flotantes y conectados directo a los módulos aritméticos.
- El procesador conserva la filosofía **RISC**: repertorio acotado.

> **Límites**
>
> Los manuales listan **5 variantes de multiplicación acumulativa (MAC)** que harían de la Pico 2 un DSP, pero **los compiladores y ensambladores actuales no permiten usarlas** desde ensamblador inline. Tampoco hay funciones trascendentales (seno, coseno, logaritmo) en hardware: se emulan por software con `math`.
>
> - En la Pico 1, `math` se emula por software (muchos ciclos); la Pico 2 aprovecha la FPU.

### 2. Instrucciones de la FPU (prefijo v)

| Instrucción | Operación |
|---|---|
| `vadd.f32 sd, sn, sm` | SD = SN + SM |
| `vsub.f32 sd, sn, sm` | SD = SN − SM |
| `vmul.f32 sd, sn, sm` | SD = SN × SM |
| `vdiv.f32 sd, sn, sm` | SD = SN / SM — **la de mayor latencia** (el manual no da los ciclos exactos) |
| `vsqrt.f32 sd, sm` | SD = √SM (solo 2 operandos) |
| `vldr sd, [rn, offset]` | Cargar en sd desde la dirección rn + offset |
| `vstr sd, [rn, offset]` | Guardar sd en rn + offset |

Accesorias: `vabs`, cambio de signo y `vcvt` (entero ↔ flotante). Las direcciones de las variables de Python llegan por los registros ARM (r0, r1…); no se pasan inmediatos directos a la FPU.

## 3. Ejercicio 1: raíz cuadrada

```
input = float(80.0); output = float(0.0)

@micropython.asm_thumb
def raiz_cuadrada(r0, r1):   # r0 = &input, r1 = &output
    vldr(s0, [r0, 0])        # s0 = 80.0
    vldr(s1, [r1, 0])        # s1 = 0.0
    vsqrt(s2, s0)            # s2 = sqrt(s0)
    vstr(s2, [r1, 0])        # output = s2

raiz_cuadrada(input, output); print(output)
```

Un valor negativo marca error (no hay imaginarios).

## 4. Ejercicio 2: sumatoria de productos (producto punto)

Σ data1[i] × data2[i]. Registros: **r0** → data1, **r1** → data2, **r2** → resultado, **r3** → contador (ambos arreglos del mismo tamaño). Acumulador s4 = 0.0.

```
label(BUCLE)
vldr(s0, [r0, 0])     # data1[i]
vldr(s1, [r1, 0])     # data2[i]
vmul(s2, s0, s1)      # producto
vadd(s4, s4, s2)      # acumular
add(r0, r0, 4)        # siguiente flotante: 4 bytes
add(r1, r1, 4)
sub(r3, r3, 1)        # contador – 1
bne(BUCLE)            # repetir si no es cero
vstr(s4, [r2, 0])     # guardar resultado
```

**Offset de 4 bytes**

La memoria es lineal en bytes y cada flotante de 32 bits ocupa **4 bytes**: los apuntadores avanzan de 4 en 4. Avanzar 1 byte lee datos corruptos.

Es la base de vectores 2D, gráficos de videojuegos en la Pico y filtros digitales.

## 5. Filtros digitales FIR e IIR

Ecuación de diferencia de un IIR (polos y ceros):

$$\mathbf{y[k] = \Sigma_{n=0}..N\ a_n \cdot x[k-n] + \Sigma_{m=1}..M\ b_m \cdot y[k-m]}$$

- Primera sumatoria: entradas presentes y pasadas (coeficientes $a_n$). Segunda: **salidas anteriores** (coeficientes $b_m$, retroalimentación).
- Un FIR solo tiene la primera. Error común: implementar un IIR sin la retroalimentación. Base de la Clase 9 y de un trabajo práctico.

## PARTE 2 · Resumen para estudiar

*Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.*

**FPU.** s0–s31 de 32 bits; RISC; sin MAC accesible desde ensamblador inline; sin funciones trascendentales.

> **Instrucciones (examen)**
>
> `vadd`, `vsub`, `vmul`, `vdiv` (la más lenta), `vsqrt` (2 operandos), `vldr`/`vstr` con [rn, offset]. Arreglos de flotantes: offset de **4 bytes**.

| Ejercicio | Clave |
|---|---|
| Raíz | r0 = &entrada, r1 = &salida; vldr → vsqrt → vstr |
| Producto punto | r0, r1 datos; r2 resultado; r3 contador; vmul + vadd en bucle; add 4; bne |
| IIR | Σ a·x[k−n] + Σ b·y[k−m] (dos sumatorias) |